Cofinanziato dall'Unione europea

REGIONE PIEMONTE

DOCENTE	Wolfgang Cecchin
UNITA' FORMATIVA	Linux Server
ARGOMENTO	find

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Introduzione

Il comando **`find`** è lo strumento principale di Linux per **cercare file e directory nel filesystem** in base a criteri molto diversi: nome, tipo, dimensione, proprietario, permessi, data di modifica e altro ancora.

A differenza di `ls`, che elenca i contenuti di una directory, `find` esplora **ricorsivamente** l'albero del filesystem a partire da un punto di inizio specificato.

1. Sintassi generale

```
$ find [percorso] [criteri] [azione]
```

Elemento	Descrizione
`[percorso]`	Punto di partenza della ricerca. Si usa `/` per tutto il filesystem, `.` per la directory corrente, `/etc` per limitare la ricerca
`[criteri]`	Uno o più filtri: nome, tipo, proprietario, permessi, dimensione, data...
`[azione]`	Cosa fare con i file trovati. Se omessa, il default è `-print` (stampa il percorso). Si può usare `-exec` per eseguire comandi

2. Criteri di ricerca principali

2.1 Nome del file (-name, -iname)

Comando	Significato
`find / -name "hosts"`	Cerca un file chiamato esattamente "hosts"
`find / -name "hosts*"`	Cerca file il cui nome inizia con "hosts" (`*` = qualsiasi suffisso)
`find / -name "*.conf"`	Cerca tutti i file con estensione `.conf`
`find / -iname "hosts"`	Come -name, ma case-insensitive (HOSTS, Hosts, hosts...)

> **NOTA:** Il carattere * è un glob (wildcard), non un'espressione regolare. Deve essere racchiuso tra virgolette per evitare che la shell lo espanda prima di passarlo a find.

2.2 Tipo di file (-type)

Opzione	Tipo cercato
`-type f`	File regolari
`-type d`	Directory
`-type l`	Link simbolici

2.3 Proprietario (-user, -group)

L'opzione `-user` filtra i file appartenenti a un utente specifico:

```
$ find / -user linda
```

Trova tutti i file nel filesystem il cui proprietario è l'utente `linda`.

```
$ find /home -group sviluppo -type f
```

Trova tutti i file regolari in `/home` appartenenti al gruppo `sviluppo`.

2.4 Dimensione (-size)

L'opzione `-size` accetta un valore con unità e un prefisso che indica il confronto:

Esempio	Significato
`find / -size +2G`	`+` = maggiore di 2 GiB
`find / -size -100M`	`-` = minore di 100 MiB
`find / -size 4k`	Esattamente 4 KiB (raro in pratica)
`find / -type f -size +100M`	Solo file regolari con dimensione superiore a 100 MiB

2.5 Data di modifica (-mtime, -mmin)

Esempio	Significato
`find /etc -mtime -1`	File modificati nelle ultime 24 ore
`find /etc -mtime +7`	File non modificati da più di 7 giorni
`find /tmp -mmin -30`	File modificati negli ultimi 30 minuti

3. Ricerca per permessi (-perm)

L'opzione `-perm` permette di cercare file in base ai loro **permessi in notazione ottale**. Esistono tre formati con significati distinti:

3.1 Tre formati di -perm a confronto

Formato	Esempio	Significato
`-perm NNNN`	`find . -perm 755`	**Corrispondenza esatta:** il file deve avere esattamente quei permessi e nessun altro
`-perm -NNNN`	`find . -perm -111`	**Tutti i bit devono essere presenti**, ma il file può averne anche altri (sottoinsieme)
`-perm /NNNN`	`find . -perm /4000`	**Almeno uno dei bit** indicati deve essere presente (OR logico)

3.2 I permessi speciali: SUID, SGID, Sticky bit

I bit speciali si specificano come quarta cifra ottale (la più significativa). Sono fondamentali per la sicurezza del sistema.

Bit	Valore ottale	Effetto
SUID	`4000`	Su file eseguibile: viene eseguito con i **permessi del proprietario del file** (es. `passwd` esegue come root)
SGID	`2000`	Su file: esegue con i **permessi del gruppo**. Su directory: i nuovi file ereditano il gruppo della directory

Sticky bit	`1000`	Su directory (es. `/tmp`): **solo il proprietario** può eliminare o rinominare i propri file, anche se la directory è scrivibile da tutti

3.3 Esempi pratici con -perm

Comando	Cosa trova
`find / -perm /4000`	File con il bit SUID attivo (formato /)
`find / -perm /2000`	File con il bit SGID attivo
`find / -type d -perm /1000`	Directory con sticky bit attivo
`find / -perm /7000`	File con almeno uno dei permessi speciali (SUID, SGID o Sticky)
`find . -perm -111`	File eseguibili da user, group e others (tutti e tre i bit x presenti)
`find . -perm -002`	File scrivibili da "others" (rischio sicurezza)
`find / -perm 755`	File con esattamente rwxr-xr-x e nessun altro bit

⚠ **ATTENZIONE:** I file con SUID o SGID attivi rappresentano un punto critico per la sicurezza. Un file SUID di proprietà di root, se vulnerabile, può essere sfruttato per ottenere privilegi di amministratore. È buona pratica censire periodicamente questi file con: find / -perm /7000 -type f 2>/dev/null

4. Azioni sui file trovati

4.1 -exec: eseguire un comando su ogni file trovato

L'opzione `-exec` permette di eseguire un comando su ciascun file trovato da `find`. La sintassi richiede due elementi obbligatori:

Elemento	Significato
`{}`	Segnaposto che viene sostituito dal percorso del file trovato in ogni iterazione
`\;`	Termina il comando `-exec`. Il backslash (\) è necessario come carattere di escape per impedire alla shell di interpretare il punto e virgola come separatore di comandi

Sintassi completa

```
$ find [percorso] [criteri] -exec comando {} \;
```

Copiare tutti i file di un utente in una directory

```
$ find / -user linda -exec cp {} /root/linda \;
```

Per ogni file di proprietà di `linda`, esegue `cp file /root/linda`. Il `{}` viene sostituito dal percorso completo del file trovato ad ogni iterazione.

Trovare file con SUID e listarli in dettaglio

```
$ find / -perm /4000 -type f -exec ls -lh {} \;
```

Trovare file in /etc che contengono una stringa e copiarli

```
$ find /etc -exec grep -l student {} \; -exec cp {} find/contents/ \; 2>/dev/null
```

Esegue due azioni concatenate: prima `grep -l` per trovare i file che contengono "student", poi `cp` per copiarli. I due `-exec` vengono eseguiti in sequenza sullo stesso file trovato.

Alternativa con + (più efficiente)

```
$ find / -user linda -exec cp {} /root/linda/ +
```

Il terminatore `+` (al posto di `\;`) raggruppa tutti i file in un'unica chiamata al comando, come farebbe `xargs`. È più efficiente ma il comando deve accettare più argomenti contemporaneamente.

4.2 Pipeline con xargs

In alternativa a `-exec`, si può usare una pipeline con `xargs` per passare i risultati di `find` a un altro comando:

```
$ find /etc/ -name '*' -type f | xargs grep "127.0.0.1"
```

Trova tutti i file regolari in `/etc/` e cerca la stringa `127.0.0.1` in ciascuno di essi. Equivale a `grep -r "127.0.0.1" /etc/` ma con il controllo aggiuntivo `-type f`.

> **NOTA:** xargs è generalmente più efficiente di -exec ... \; perché raggruppa gli argomenti in meno chiamate al comando. Tuttavia, con nomi di file contenenti spazi è necessario usare: find ... -print0 | xargs -0 comando

4.3 -printf: output personalizzato

```
$ find /etc -name '*ini' -printf '%s, %p\n' | sort -rn
```

Mostra dimensione in byte (`%s`) e percorso (`%p`) di ogni file `.ini` trovato, ordinati per dimensione decrescente (`sort -rn`).

5. Sopprimere gli errori con 2>/dev/null

Quando si esegue `find` a partire dalla radice del filesystem (`/`), si incontrano spesso directory non accessibili all'utente corrente, come `/proc`, `/sys` o directory di altri utenti. Questi accessi negati generano messaggi di errore su **standard error (stderr)** che rendono l'output difficile da leggere.

Il reindirizzamento `2>/dev/null` scarta tutti i messaggi di errore:

Elemento	Valore	Significato
`2`	stderr	File descriptor 2 = standard error (messaggi di errore)
`>`	redirige	Reindirizza l'output verso...
`/dev/null`	cestino	Dispositivo speciale che scarta tutto ciò che riceve

Forma tipica

```
$ find / -name "hosts" 2>/dev/null
```

Cerca il file "hosts" in tutto il filesystem, ignorando i messaggi "Permission denied".

```
$ find / -perm /4000 -type f 2>/dev/null
```

Trova tutti i file SUID nel sistema, senza mostrare gli errori di accesso negato.

> **Buona pratica** Aggiungere 2>/dev/null quasi sempre quando si usa find su / o su directory di sistema. La ricerca resta funzionale e l'output è pulito.

6. Escludere percorsi con ! -path

Per escludere specifici percorsi dalla ricerca si usa `!` (NOT logico) insieme a `-path`:

```
$ find / -name "student" -type f ! -path '*/proc/*' ! -path '*/tmp/*'
```

Cerca file chiamati "student" in tutto il filesystem, escludendo le directory `/proc` e `/tmp`. Questo è utile per evitare i filesystem virtuali e le directory temporanee che raramente contengono i file di interesse.

7. Riepilogo delle opzioni principali

Opzione	Descrizione
`-name "pattern"`	Cerca per nome (case-sensitive, supporta glob *)
`-iname "pattern"`	Come -name ma case-insensitive
`-type f|d|l`	Filtra per tipo: file (f), directory (d), link simbolico (l)
`-user nome`	File appartenenti all'utente specificato
`-group nome`	File appartenenti al gruppo specificato
`-size +N[kMG]`	File di dimensione maggiore di N. Unità: `k`=KiB, `M`=MiB, `G`=GiB
`-mtime -N`	File modificati negli ultimi N giorni
`-perm NNNN`	Permessi esatti
`-perm -NNNN`	Tutti i bit indicati presenti (altri possibili)
`-perm /NNNN`	Almeno uno dei bit indicati presente
`-exec cmd {} \;`	Esegue un comando su ogni file trovato
`-printf 'formato'`	Stampa l'output in formato personalizzato
`! -path '*/dir/*'`	Esclude un percorso dalla ricerca
`2>/dev/null`	Sopprime i messaggi "Permission denied"

8. Esercizi

Ricerca base

Es. 1 — Cercare in tutto il filesystem un file di nome `resolv.conf`, sopprimendo gli errori.

Es. 2 — Trovare tutti i file con estensione `.log` nella directory `/var/log`.

Es. 3 — Trovare tutte le **directory** che si chiamano `backup` a partire da `/home`.

Es. 4 — Trovare i file di proprietà del tuo utente in `/tmp`.

Ricerca per dimensione e data

Es. 5 — Trovare tutti i file regolari di dimensione superiore a 100 MiB nel filesystem.

Es. 6 — Trovare i file in `/etc` modificati nelle ultime 24 ore.

Permessi

Es. 7 — Trovare tutti i file nel sistema con il bit SUID attivo. Aggiungere `2>/dev/null` per un output pulito.

Es. 8 — Trovare le directory con lo sticky bit attivo (come `/tmp`).

Es. 9 — Trovare tutti i file con almeno uno dei permessi speciali (SUID, SGID, Sticky) attivi.

Es. 10 — Trovare i file scrivibili da "others" in `/home` (potenziale rischio sicurezza).

Azioni e pipeline

Es. 11 — Trovare tutti i file di configurazione (`*.conf`) modificati negli ultimi 7 giorni in `/etc` e visualizzarli con `ls -lh` usando `-exec`.

Es. 12 — Cercare in `/etc/` tutti i file che contengono la stringa `127.0.0.1` usando `find` + `xargs grep`.

Es. 13 — Trovare tutti i file di proprietà dell'utente `mario` e copiarli in `/root/backup_mario/`, usando `-exec`.

Es. 14 (avanzato) — Trovare file chiamati `student` nel filesystem, escludendo `/proc` e `/tmp`, sopprimendo gli errori.